CS61C Summer 2025 — Yokota Final

Solutions last updated: Monday, August 18, 2025

PRINT Your Name: _____

PRINT Your Student ID: _____

PRINT the Name and Student ID of the person to your left: _____

PRINT the Name and Student ID of the person to your right: _____

PRINT the Name and Student ID of the person in front of you: _____

PRINT the Name and Student ID of the person behind you: _____

You have 170 minutes. There are 8 questions of varying credit. (100 points total)

Question:	1	2	3	4	5	6	7	8	Total
Points:	19	20	14	21	7	9	10	0	100

For questions with **circular bubbles**, you may select only one choice.

- ○ Unselected option (Completely unfilled)
- ◌ Don't do this (it will be graded as incorrect)
- ● Only one selected option (completely filled)

For questions with **square checkboxes**, you may select one or more choices.

- ☐ You can select
- ☐ multiple squares
- ☑ (Don't do this)

Anything you write outside the answer boxes or you ~~cross out~~ will not be graded. If you write multiple answers, your answer is ambiguous, or the bubble/checkbox is not entirely filled in, we will grade the worst interpretation. For coding questions with blanks, you may write at most one statement per blank and you may not use more blanks than provided.

If an answer requires hex input, you must only use capitalized letters (`0xB0BACAFE` instead of `0xb0bacafe`). For hex and binary, please include prefixes in your answers unless otherwise specified, and do not truncate any leading 0's. For all other bases, do not add any prefixes or suffixes.

As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others. I will follow the rules of this exam.

Acknowledge that you have read and agree to the honor code above and sign your name below:

_____

Q1 *Potpourri Pot-pouring* 🪔 **(19 points)**

Q1.1 (3 points) Convert the 8-bit number `0xF3` into decimal, interpreted in each of the formats below:

Unsigned:	**Sign-Magnitude:**	**Two's Complement:**
243	-115	-13

Solution: To find the unsigned representation, we convert `0xFF3` into binary: `0b1111 1111 0011`. Then, we take the sum

$$2^7 + 2^6 + 2^5 + 2^4 + 2^1 + 2^0 = 243.$$

To find the sign-magnitude equivalent, we notice that we we have the same number, but without the MSB's place value, as the MSB now signifies a negative number. So, we can compute

$$-(243 - 128) = -115.$$

Lastly, to find Two's Complement, we notice that the place value of the MSB has gone from $+2048$ to -2048, or a change in value by -4096. We can thus compute

$$243 - 256 = -13.$$

For Q1.2–Q1.3, suppose you have a IEEE-754 standard floating-point format with 1 sign bit, 15 exponent bits (with a standard bias of −16,383), and 16 mantissa bits. For each of the below ranges, how many distinct floating point numbers are representable within them?

Q1.2 (2 points) $[1, 16)$[1]. Express your answer in terms of powers of two.

2^{18}

Solution: Notice that for any fixed exponent, we have exactly $2^{\#\text{ mantissa bits}} = 2^{16}$ possible unique numbers. Now, we can divide our range into 4 disjoint ranges, each with a unique fixed mantissa:

$$\begin{aligned}[1, 16) = [1, 2) &\quad (\text{Exponent} = 0)\\ \cup\, [2, 4) &\quad (\text{Exponent} = 1)\\ \cup\, [4, 8) &\quad (\text{Exponent} = 2)\\ \cup\, [8, 16) &\quad (\text{Exponent} = 3)\end{aligned}$$

Since each of these 4 disjoint ranges contains 2^{16} distinct values, in total, we have $4 \times 2^{16} = \mathbf{2^{18}}$ distinct values.

[1] $[x, y)$ denotes the range from x to y, including x and excluding y. i.e. the set of all z such that $x \leq z < y$.

Q1.3 (2 points) $[x, y)$, where x and y are the numbers encoded by `0x61BC0DED` and `0x61CC0DED` respectively. Express your answer as an unsigned hexadecimal integer.

`0x100000`

Solution: While it is possible to repeat the previous analysis after first converting these numbers to decimal representations, we can notice a property of floating point that is helpful here: the finite positive floating point numbers are ordered in the same way as the unsigned numbers. Since both of the values here are positive, we can simply take

$$\texttt{0x61CC0DED} - \texttt{0x61BC0DED} = \texttt{0x100000}$$

Q1.4 (1 point) Consider the following statements about CALL. Which are true? Select all that apply.

■ The compiler is responsible for choosing which registers different variables are stored in.

B The assembler is responsible for choosing which registers different variables are stored in.

C The linker is responsible for linking virtual pages to physical pages.

■ The loader is responsible for populating the code segment of memory for each program.

E None of the above

Solution:
- The compiler is responsible for register allocation, while the assembler is only responsible for converting instructions into machine code, meaning that the first choice is true and the second choice is false.
- The linker is responsible for linking together different files, not virtual memory, making the third choice false.
- The loader loads a program into memory and redirects execution to it, making the last choice true.

Q1.5 (2 points) Convert the following RISC-V machine code into its corresponding instruction. Express immediates in decimal and use the corresponding register names instead of numbers (i.e. `s5` instead of `x21`).

`0x408FDF33` ➡

Solution: `sra t5 t6 s0`

Q1.6 (2 points) What is the tag-index-offset breakdown for a 1 MiB, 8-way set-associative cache with 64 B blocks, on a system with a 512 GiB address space?

T: 22 bit(s)	I: 9 bit(s)	O: 8 bit(s)

Solution: Starting with offset, we find that

$$\#\text{ offset bits} = \log_2(\text{block size}) = \log_2(256) = \mathbf{8}.$$

Next, we see that our cache is 1MiB in size, meaning that we have $\frac{1\text{MiB}}{256\text{B}} = 4\text{Ki}$ blocks. Since this is an 8-way set associative cache, each index holds 8 blocks, meaning that we have $\frac{4\text{Ki blocks}}{8\frac{\text{blocks}}{\text{index}}} = 512$ indices. Now, we take

$$\#\text{ index bits} = \log_2(\#\text{ indices}) = \log_2(512) = \mathbf{9}.$$

Lastly, to find the number of tag bits, we first find the length of a memory address:

$$\#\text{ address bits} = \log_2(\#\text{ bytes in memory space}) = \log_2(512\text{ Gi}) = 39.$$

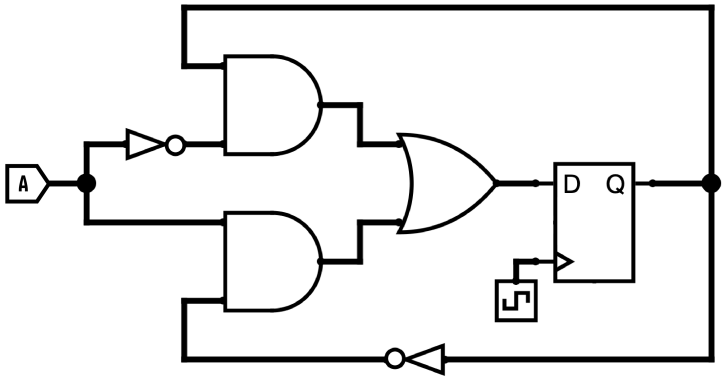
Since we know that

$$\#\text{ Tag Bits} + \#\text{ Index Bits} + \#\text{ Offset Bits} = \#\text{Memory Address Bits},$$

we can compute

$$\#\text{ Tag Bits} + 9 + 8 = 39 \Rightarrow \#\text{ Tag Bits} = \mathbf{22}.$$

For Q1.7–Q1.8, consider the following circuit and timings.



$t_{\texttt{setup}} = 5$ ns
$t_{\texttt{clk-to-q}} = 10$ ns
$t_{\texttt{NOT}} = 10$ ns
$t_{\texttt{AND}} = 20$ ns
$t_{\texttt{OR}} = 15$ ns

You may assume that:
- Registers start initialized to zero
- `A` is connected to the output of a register.

Q1.7 (1 point) What is the minimum clock period this circuit can have to consistently exhibit well-defined behavior?

60ns

Solution: For this solution, we are starting from the output of `A` or the Q end of the register, and ending at the D end of the register.

$$\begin{aligned} t_{\texttt{clk-period}} &\geq t_{\texttt{clk-to-q}} + t_{\texttt{longest-combinational-delay}} + t_{\texttt{setup}} \\ &= t_{\texttt{clk-to-q}} + t_{\texttt{NOT}} + t_{\texttt{AND}} + t_{\texttt{OR}} + t_{\texttt{setup}} \\ &= 10 + 10 + 20 + 15 + 5 \\ &= 60\text{ns} \end{aligned}$$

Q1.8 (1 point) What is the maximum hold time this circuit can have to consistently exhibit well-defined behavior?

45ns

Solution: The shortest combinational path in this circuit involves just a single AND gate and a single OR gate, and can start at either input `A` or the D end of the register, and ends at the Q end of the register. Using the formula for hold time, we have

$$\begin{aligned} t_{\texttt{hold-time}} &\leq t_{\texttt{clk-to-q}} + t_{\texttt{shortest-combinational-delay}} \\ &= t_{\texttt{clk-to-q}} + t_{\texttt{AND}} + t_{\texttt{OR}} \\ &= 10 + 20 + 15 \\ &= 45\text{ns} \end{aligned}$$

Q1.9 (2 points) Joshua has a program that has a runtime of 200 minutes, 70% of which occurs in function `foo`. After parallelizing `foo`, he finds that the program now has a runtime of 100 minutes. How many times faster does `foo` now run?

3.5× faster

Solution: We find that Joshua obtains a 2 times speedup by computing $\frac{200 \text{ minutes}}{100 \text{ minutes}} = 2$. Knowing this, we can now use Amdahl's law:

$$2 = \frac{1}{(1 - 0.7) + \frac{0.7}{N}} = \frac{1}{0.3 + \frac{0.7}{N}}.$$

Solving for N, we find the solution: $N = 3.5$.

Q1.10 (3 points) Which of the following are responsibilities of most operating systems? Select all that apply.

- [A] Compiling programs
- [■] Loading processes
- [C] Managing the L1 cache
- [■] Managing virtual memory
- [■] Managing a program's I/O
- [■] Regulating memory use of processes
- [G] Assigning SIMD vectors to CPU cores running SIMD code
- [■] Managing multiple processes and assigning them to cores
- [I] Interpreting programs written in assembly languages
- [J] None of the above

Solution:

- A compiler is responsible for compiling programs, which is not built into the OS.
- The loader is part of the operating system.
- The L1 cache is managed by hardware, not the OS.
- Virtual memory requires the OS and the MMU to work together.
- I/O is handed at an OS level
- Memory limits are assigned by the OS to processes.
- SIMD registers are assigned in the same way as other registers — by the compiler
- The OS schedules processes to run across multiple cores
- To interpret a program written in assembly language, we either use a simulator or directly assemble to the program to be run on hardware.

Q2 *Cheap C-heap* 💸 **(20 points)**

Noah is curious about how the heap works, and decides to try and implement the heap himself. To set this up, he creates the following class to store `malloc` metadata in a doubly-linked list structure:

```
1 typedef struct mem_meta {
2   struct mem_meta *prev; // Pointer to metadata of previous allocation
3   struct mem_meta *next; // Pointer to metadata of next allocation
4
5   // Number of bytes in this allocation, INCLUDING the bytes of this mem_meta
6   size_t total_size;
7 } mem_meta;
```

Q2.1 (1 point) On a 32-bit system, what is `sizeof(mem_meta)`?

12 bytes

Solution: On a 32-bit system, all pointer and `size_t` take up 4 bytes of space. Since we have 2 pointers and one `size_t`, `sizeof(mem_meta) = 12`.

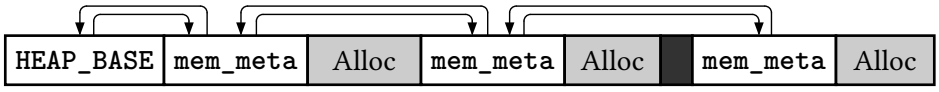
Next, we will implement the `malloc` function according to the spec below:

`malloc`: Allocates `size` bytes of memory and returns a pointer to the allocated memory. Returns `NULL` if allocation fails for any reason. If there is a free block large enough to fit the requested space, `malloc` will use that block. Otherwise, `malloc` grows the heap to create the requested space.		
Arguments	`size_t size`	The number of bytes to allocate.
Return value	`void *`	A pointer to the allocated memory.

As an example, consider the following heap state:

If we request a large `malloc` that would not fit in the free space, then `malloc` would grow the heap and return new space appended to the heap.

Alternatively, a small `malloc` that could fit in the free block would result in the following heap state:



For the rest of this question, you may use the following symbols:

`const mem_meta *HEAP_BASE`	A sentinel `mem_meta`, which acts as the start of the heap. This `mem_meta` will never have `free` called on it, has a `prev` of `NULL`, and `total_size == sizeof(mem_meta)`.
`void *sbrk(int inc)`	Requests a change in heap size from the OS. • If `inc >= 0`, increases the size of the heap by `increment` bytes, with the new space writable.

	• If `inc < 0`, decreases the size of the heap by **`increment`** bytes. • Returns a pointer to the end of the heap as it was before calling **`sbrk`**, or `(void*) -1` if the OS refuses to allocate more memory.

(11 points) Complete the function `malloc`, as described below.

```
1  void *malloc(size_t size) {
2    mem_meta *curr = HEAP_BASE;
                      Q2.2
3    size_t required_space = sizeof(mem_meta) + size;
                                    Q2.3
4    while(curr->next) { // Search for free space between blocks
            Q2.4
5      void *first_free_byte = (char *) curr + curr->total_size;
                                          Q2.5
6      if((char *) curr->next - (char *) first_free_byte >= required_space) {
7        mem_meta *new_block = first_free_byte;
                               Q2.6
8        new_block->prev = curr;
9        new_block->next = curr->next;
10       new_block->total_size = required_space;
11       curr->next->prev = new_block;
                  Q2.7
12       curr->next = new_block;
                Q2.8
13       return new_block + 1;
                  Q2.9
14     }
15     curr = curr->next;
             Q2.10
16   }
17   //Unable to find space; attempt to increase size of heap, return NULL if unsuccessful
18   mem_meta *new_tail = sbrk(required_space);
                                  Q2.11
19   if(new_tail == (void *) -1) { return NULL; }
               Q2.12
20   new_tail->prev = curr;
21   new_tail->next = NULL;
22   new_tail->total_size = required_space;
23   curr->next = new_tail;
           Q2.13
24   return new_tail + 1;
              Q2.14
25 }
```

(Question 2 continued...)

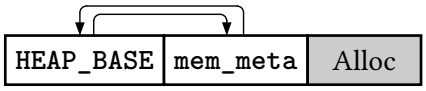
(5 points) Complete the function **free**, as described below.

free: Deallocates the allocation pointed to by **ptr**. If **ptr** is a **NULL** pointer, no operation is performed. Adjacent free blocks are merged together. If the last block is freed, the size of the heap is reduced.		
Arguments	`void *ptr`	Memory to deallocate. Must be a pointer returned by **malloc**.
Return value	`void`	

Consider the heap state from the previous page (repeated here).

If we were to **free** the first allocation, we would expect the free blocks to merge together:

If we were to instead **free** the last allocation, we would expect a reduction in the size of the heap:




```
1  void free(void *ptr) {
2    if (!ptr) { return; }

3    mem_meta *metadata = ((mem_meta *) ptr) - 1;
                                      Q2.15
4    mem_meta *prev = metadata->prev;
5    mem_meta *next = metadata->next;

6    prev->next = next;
            Q2.16
7    if ( next ) { // Freeing a pointer in the middle of the heap
          Q2.17

8      next->prev = prev;
              Q2.18
9    } else { // Freeing the last pointer in the heap

10     char *curr_heap_end = (char *) metadata + metadata->total_size;
                                                    Q2.19

11     char *new_heap_end = (char *) prev + prev->total_size;
                                           Q2.20
12     sbrk(-(curr_heap_end - new_heap_end));
13   }
14 }
```

Q2.21 (3 points) If a pointer is **free**'d in the middle of an allocated block, what errors would be caused in the design above? *Note*: segmentation faults do not occur when accessing free bytes within the heap.

Solution: The data immediately before the pointer will be treated as if it was a valid **mem_meta**, which likely causes a segmentation fault trying to access garbage **next** and **prev** pointers.

Q3 *K-pop Ka-pop* **(14 points)**

Alex has been called in to help fix Alonzo's implementation of the single-cycle datapath given on the reference card. Lightning McQueen runs the following code on Alonzo's CPU:

```
1 addi t0 x0 4       # Time Step 1: Sets t0 to 4
2 jal ra 8           # Time Step 2: Jump to instruction 4
3 beq x0 x0 safe     # Time Step 6: Branches to safe because 0 == 0
4 beq x0 t0 kaboom   # Time Step 3: Does not branch because 0 != 4
5 addi t0 t0 -2      # Time Step 4: Sets t0 to 2 (was clarified to be 2, not 0)
6 jr ra              # Time Step 5: Jumps to instruction 3
```

On a correctly implemented CPU, this code should result in safe execution (executing the instruction located at the label `safe` instead of the instruction located at `kaboom`, regardless of the location of the respective labels), as noted by the comments. However, **Alonzo's CPU executed the instruction at `kaboom` without executing the instruction at `safe`** — McQueen somehow ended up getting gapped and KaChow went KaBOOM!

After investigating, Alex discovers that one of the control signals was hardcoded to a fixed value in the CPU. There were three possible control signals that, when hardcoded, could have caused `kaboom` to be executed before `safe`: `PCSel`, `RegWEn`, and `WBSel`. For Q3.1–Q3.3, select the value that the indicated control signal would need to have been hardcoded to, and then indicate the sequence of lines that get executed before `kaboom` is reached.

For example, the execution order on a fully correct CPU would be `1, 2, 4, 5, 6, 3, safe`.

You may assume that:

- The code segment starts at address `0x0000 0000`
- Each part is independent of each other, and for any given subpart, all other control signals are correct.
- All registers besides `sp`, `gp`, and `tp` are initialized to 0.
  - The specific values in `sp`, `gp`, and `tp` are not relevant to solving this problem.

Q3.1 (2 points) `PCSel`

Ⓐ PC + 4 ● ALU

`1, 2, 4, kaboom`

Q3.2 (2 points) `RegWEn`

● Disabled Ⓑ Enabled

`1, 2, 4, kaboom`

Q3.3 (2 points) `WBSel`

Ⓐ Mem ● ALU Ⓒ PC + 4

`1, 2, 4, 5, 6, 4, 5, 6, 4 kaboom`

After fixing this CPU, Saja Boys star Jinu wants to make Alonzo's single cycle CPU more demonic by implementing a new instruction called **`sodapop rd rs1 rs2`**. If **`rs1 < rs2`**, this instruction will POP the value at **`Mem[rs2]`** into **`rd`**, and then StOre the DAta at **`rs2`** to **`Mem[rs2]`**. That is, the **`sodapop`** follows the below description:

```
if (rs1 < rs2) { // signed comparison
  rd = Mem[rs2]
  Mem[rs2] = rs2
}
```

Q3.4 (3 points) What minimal additional changes, if any, would we need to make to our single-cycle datapath in order for us to implement `sodapop` (with as few changes as possible)? Select all that apply.

- [A] Create a new instruction type and update the ImmGen
- [B] Add a new read input to the RegFile for a third register value
- [C] Add a new WriteData and WriteIndex input to the RegFile
- [D] Add a third possible value for `ASel` and update the corresponding MUX/control logic
- [E] Add a third possible value for `BSel` and update the corresponding MUX/control logic
- [F] Add a new ALU operation and update any relevant selector/control logic.
- [G] Add a third possible value for `PCSel` and update the corresponding MUX/control logic
- [■] Allow the DMEM to be able to read and write at the same clock cycle and update any relevant selector/control logic
- [I] Add a new read input to DMEM for a second memory read output
- [J] Add a fourth possible value for `WBSel` and update the corresponding MUX/control logic
- [K] None of the above

Solution: The `Mem[rs2] = rs2` part of the instruction requires no extra hardware to perform since `MemAddress` is computed by the ALU, which we can set to perform `Bsel` as the operation instead of `add` (as used in `lui`).

However, we also need to read `Mem[rs2]` before we overwrite it with our new value, so we want to be able to read memory **and then** write to it.

As discussed in lecture, reads are executed in the **same clock cycle** whereas memory writes are updated in the **following** cycle — this means that allowing reads and writes in the same clock cycle allows for a write to be executed immediately after the value is read. This is not possible if we do not modify DMEM to be able to read and write in the same clock cycle, so that modification is necessary.

The comparison `rs1 < rs2` can be evaluated using the branch comparator (BrEq, BrLt, and BrUn) outputs in the Control Logic. If the condition is not met, we can use `RegWEn` and `MemRW` to prevent the instruction from updating `rd` or `Mem[rs2]`, essentially turning it into a `noop` if the condition is not met, without requiring any additional hardware.

HUNTR/X star Rumi wants to show Jinu "how it's done" and pipelines Alonzo's datapath! The rest of this question uses the 5-stage pipelined datapath provided on the reference card. Consider the following code:

```
1 hazardous: addi t0 t1 5
2            beq t1 t0 hazardous
3            addi t1 t0 5
4            addi a1 a1 4
```

Assuming that the CPU is completely unoptimized (i.e. no double pumping, no branch prediction, no forwarding paths, all hazards resolved via stalling), which hazards occur when the above program is run on the 5-stage pipeline?

For each hazard, indicate which instruction causes the hazard, the instruction that is stalled due to the hazard, and the type of hazard that occurred. If a single instruction would cause multiple following instructions to be stalled, list each such hazard independently. You may not need all the given blanks; if you believe that there are less than 5 hazards, select "N/A" and leave the boxes blank.

Order the hazards by line number of the originating instruction, then the affected instruction.

Q3.5 (1 point) Hazard 1:

Between instructions [1] and [2]

● Data Ⓒ Structural
Ⓑ Control Ⓓ N/A

Q3.6 (1 point) Hazard 2:

Between instructions [1] and [3]

● Data Ⓒ Structural
Ⓑ Control Ⓓ N/A

Q3.7 (1 point) Hazard 3:

Between instructions [2] and [3]

Ⓐ Data Ⓒ Structural
● Control Ⓓ N/A

Q3.8 (1 point) Hazard 4:

Between instructions [2] and [4]

Ⓐ Data Ⓒ Structural
● Control Ⓓ N/A

Q3.9 (1 point) Hazard 5:

Between instructions [ ] and [ ]

Ⓐ Data Ⓒ Structural
Ⓑ Control ● N/A

Q4 *Double Double Double* 😈 **(21 points)**

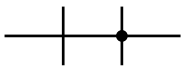
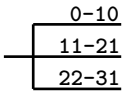
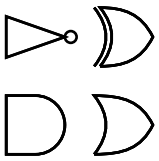
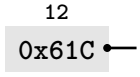
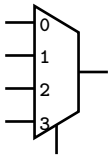
We will build a circuit that implements `double_double`, defined below:

`double_double`: Doubles a `double`.		
Arguments	`In (64 bits)`	An IEEE_754 **double-precision** floating point number
Return value	`Out (64 bits)`	The `double` representing `2 * x`.

A C implementation of `double_double` has been provided below:

```
1 #define INTINF (0x7FF << 52) // bitwise representation of INFINITY
2 uint64_t double_double(uint64_t x) {
3   switch((x >> 52) & 2047) {
4     case 0:    return (x << 1) | (x & (1 << 63));
5     case 2047: return x;
6     case 2046: return INTINF | (x & (1 << 63));
7     default:   return x + (1 << 52);
8   }
9 }
```

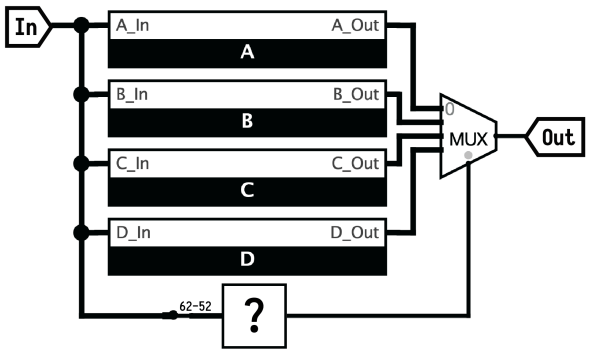
You may use the following components in your circuit:

Name	Image	Description
Wire		Transmits data. Can be multi-bit. For this entire question, bit 0 is the **LSB** of a wire. Intersecting wires must have a circle drawn where they cross. Wires without intersections will be treated as unconnected.
Splitter		Splits or merges wires. The example given receives 32 input bits and splits into bits `0-10` on top, `11-21` in the middle, and `22-31` at the bottom. Splitters may be defined with different fan outs.
Logic Gates		Performs the appropriate bitwise operation. Can receive multiple-bit inputs, and for AND and OR, more than two inputs.
Constants		The constant is written in hexadecimal. Must specify the bit width of the constant above the block. You may write `INTINF` to represent the 63-bit constant `0x7FF0 0000 0000 0000`.
Multiplexer		Outputs the top input if the selector is 0, the second-from top if the selector is 1, etc. Can be multi-bit (e.g. 2 selector bits and 4 inputs).
Incrementer		Increments the input by 1 (treating the input as an int).
Control		Receives as input an 11-bit value, and outputs a 2-bit value: • If the input is `0x000`, outputs `0b00` • If the input is `0x7FF`, outputs `0b01` • If the input is `0x7FE`, outputs `0b10` • For any other input, outputs `0b11`

(Question 4 continued…)

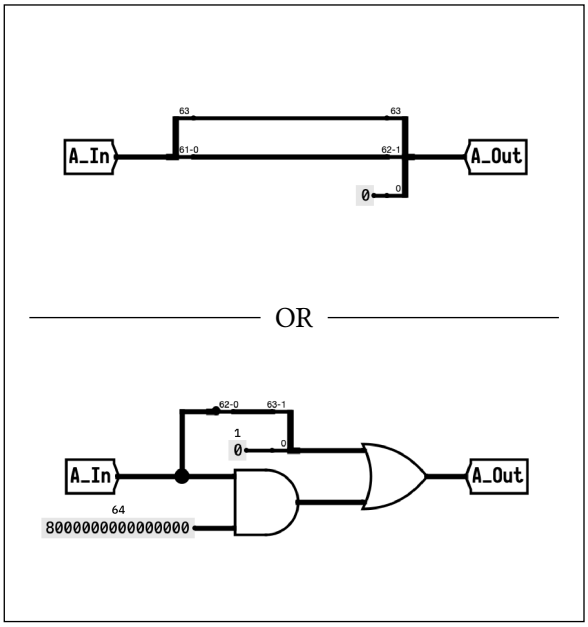
The circuit to the right is one possible implementation that correctly computes `double_double`. It has 4 subcircuits, corresponding to each arm of the `switch` statement:

- Subcircuit `A` should compute `case 0:`
- Subcircuit `B` should compute `case 2047:`
- Subcircuit `C` should compute `case 2046:`
- Subcircuit `D` should compute `default:`

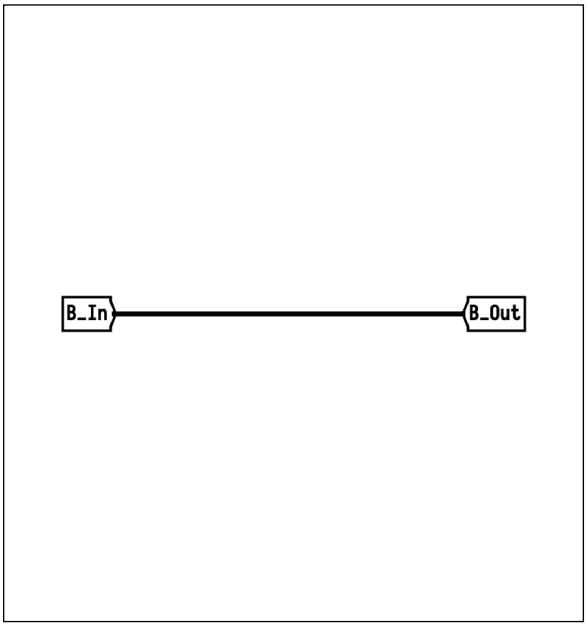


Q4.1 (8 points) Fill in each subcircuit to complete a circuit that computes `double_double`.

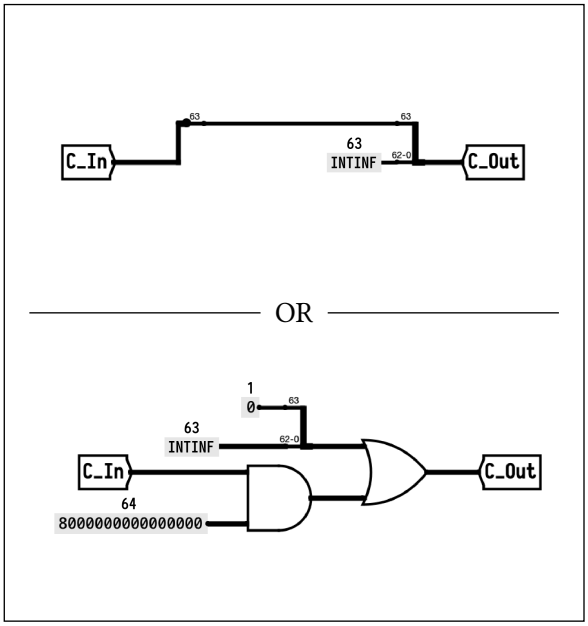
Subcircuit `A`[2]



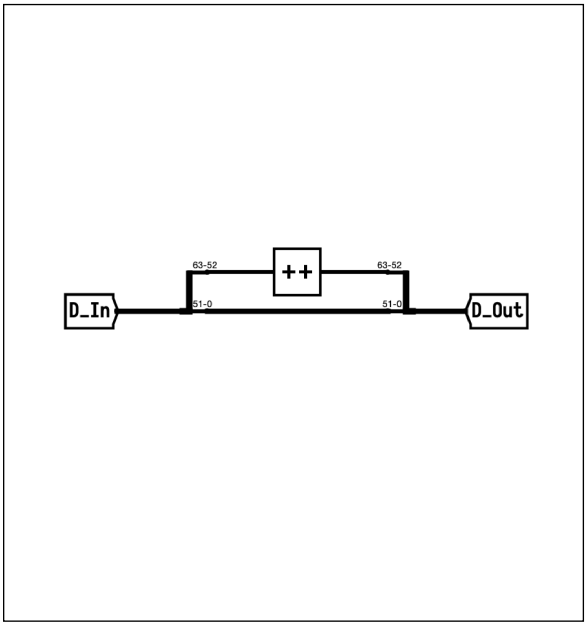
Subcircuit `B`



Subcircuit `C`



Subcircuit `D`



[2]There are more possibilities than just these solutions, but the two main approaches are given for subcircuits `A` and `C`

Q4.2 (4 points) The Control circuit receives an eleven-bit input `0b ABC DEFG HJKL`, and outputs two bits, as defined below:

- If the input is `0x000`, outputs `0b00`
- If the input is `0x7FF`, outputs `0b01`
- If the input is `0x7FE`, outputs `0b10`
- For any other input, outputs `0b11`

However, you notice that you can simplify your truth table by first computing bits

```
X = A | B | C | D | E | F | G | H | J | K
Y = A & B & C & D & E & F & G & H & J & K
```

Fill out the truth table to the right for `Control`, using bits `X`, `Y`, and `L`. If a particular state is impossible, write "N/A".

X	Y	L	Out[1]	Out[0]
0	0	0	0	0
0	0	1	1	1
0	1	0	N/A	N/A
0	1	1	N/A	N/A
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	0	1

For Q4.3–Q4.4, write Boolean expressions for the indicated value in terms of `X`, `Y`, and `L`. For full credit, the number of logic gates you use must be at most the indicated value. Partial credit will be awarded for solutions that use more gates than the par score, but will not be awarded if your expression yields a wrong result on any case. Your expression may evaluate to either 0 or 1 for impossible states.

Your answers may consist of the following operators and symbols:

Operators			Symbols		
NOT	AND	OR	Inputs	Constants	Parentheses
`~X`	`X & Y`	`X | Y`	`X, Y, L`	`0, 1`	`()`

Q4.3 (2.5 points) `Out[1]` (5 gates)

```
Out[1] = (L & ~Y) | (X & ~L)
```

Q4.4 (2.5 points) `Out[0]` (3 gates)

```
Out[0] = L | (X & ~Y)
```

Grading: 0 points were awarded if the equation did not match the solution above.

Otherwise, the score given was $\min\left(\frac{2.5x}{n}, 2.5\right)$, where x was the number of gates used and n is the specified number of gates.

Warning: We consider the following subpart to be harder than most other questions on the exam.

You notice that if you change the mapping from switch cases to mux selector bits specified in `Control`, you can reduce your circuit size significantly.

Q4.5 (4 points) Choose a different mapping to use for `Control`, and write Boolean expressions for the resulting output. You may use bits X and Y without incurring an operator cost. (Par 3)

- If the input is `0x000`, output `0b00`
- If the input is `0x7FF`, output `0b11`
- If the input is `0x7FE`, output `0b10`
- For any other input, output `0b01`

`Out[1]` (3 gates total for both `Out[1]` and `Out[0]`)

```
Out[1] = Y
```

`Out[0]` (3 gates total for both `Out[1]` and `Out[0]`)

```
Out[0] = L | (X & ~Y)
```

Solution: Grading note: 0 points were awarded if any of the following were true:
- Either `Out[1]` or `Out[0]` were left blank
- The four cases were not assigned unique 2-bit values of were left blank
- The four cases were all assigned the same 2-bit values as in 4.2
- The equations in `Out[1]` or `Out[0]` did not match the specified mapping

Otherwise, the score given was $4 \cdot \frac{3+x}{6}$, where x was the number of gates used.

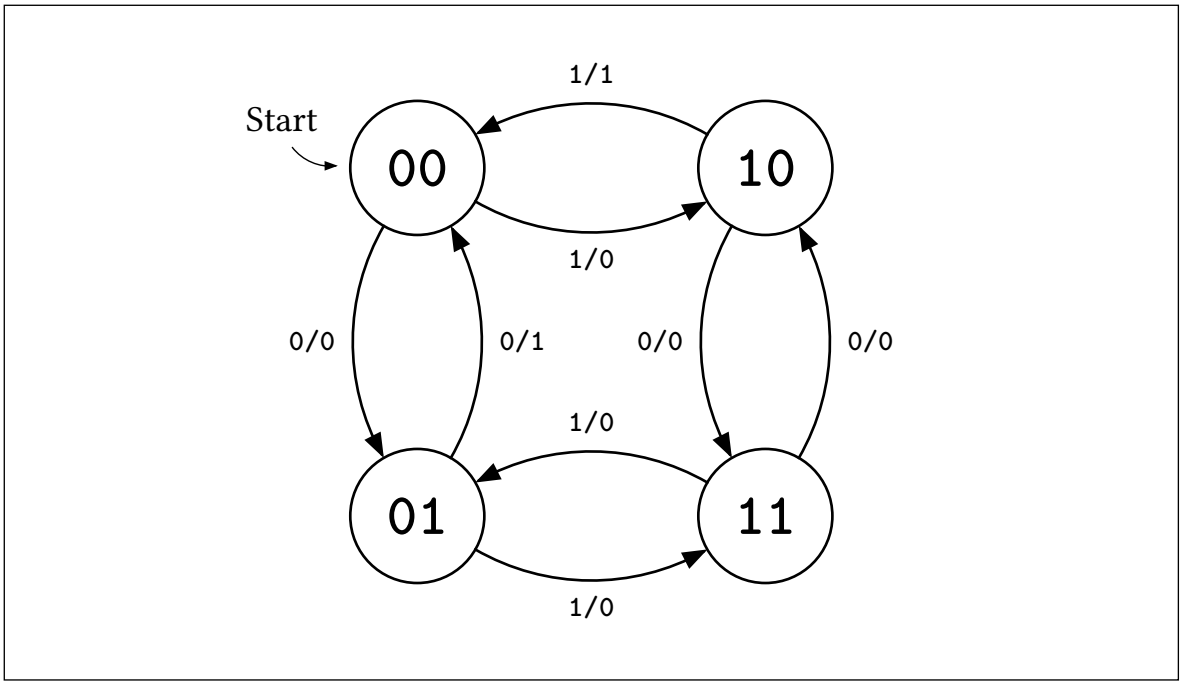
Q5 *Even Even* ⚖ **(7 points)**

We want to make an FSM that outputs `1` if the total number of `0`s and the total number of `1`s seen so far are both even, and `0` otherwise. For example, the input `110100010101` should yield an output of `010000010001`, as shown in the table below.

Example Input	`110100010101`
Example Output	`010000010001`

Input bits so far	**#0s seen**	**#1s seen**	**FSM Output**
`1`	0	1	0
`11`	0	2	1
`110`	1	2	0
`1101`	1	3	0
`11010`	2	3	0
`110100`	3	3	0
`1101000`	4	3	0
`11010001`	4	4	1
`110100010`	5	4	0
`1101000101`	5	5	0
`11010001010`	6	5	0
`110100010101`	6	6	1

Q5.1 (7 points) Implement the FSM below by filling in state transitions between the given states:



Q6 *Absalom, Ablasum!* 👑 **(9 points)**

(7 points) Complete the function `sum_abs_values` using SIMD vector instructions.

`sum_abs_values`: Computes the sum of the absolute values of an array.		
Arguments	`int32_t *arr`	An array of signed 32-bit integers.
	`int32_t n`	The number of elements in the array. You may assume that `n` is a multiple of 16.
Return value	`int32_t`	The sum of the absolute value of all values in the array. You may assume that the sum will not overflow a 32-bit signed integer.

You have access to the following SIMD operations. A single vector is a 512-bit vector register capable of holding sixteen 32-bit integers. **You may use at most one operation per blank.**

- `vector vec_load(int32_t *A)`: Loads 4 integers at memory address `A` into a vector.
- `vector vec_setnum(int32_t num)`: Creates a vector where every element is `num`.
- `vector vec_cmpgt(vector A, vector B)`: Computes `A > B` elementwise. Result is `0xFFFFFFFF` if true, 0 otherwise.
- `vector vec_and(vector A, vector B)`: Computes bitwise AND elementwise.
- `vector vec_add(vector A, vector B)`: Computes `A + B` elementwise.
- `vector vec_mul(vector A, vector B)`: Computes `A * B` elementwise.
- `int32_t vec_sum(vector A)`: Adds all elements of the vector and returns the scalar sum.

```
1  int32_t sum_abs_values(int32_t *arr, int32_t n) {
2    vector sum_vec = vec_setnum(  0  );
                                 Q6.1
3    vector ref_vec = vec_setnum(0);
4    vector neg_two_vec = vec_setnum(-2);
5    for (int32_t i = 0; i < n; i += 16) {
                         Q6.2
6      vector input_vec = vec_load(arr + i);
7      vector mask_vec = vec_cmpgt(ref_vec, input_vec);
                                   Q6.3       Q6.4
8      vector correction_vec = vec_mul(neg_two_vec, input_vec);
                                              Q6.5
9      vector masked_corr_vec = vec_and(correction_vec, mask_vec);
                                          Q6.6           Q6.7
10     vector abs_vec = vec_add(input_vec, masked_corr_vec);
11     sum_vec = vec_add(sum_vec, abs_vec);
                                 Q6.8
12   }
13   return vec_sum(sum_vec);
                    Q6.9
14 }
```

Note: The following problem is independent of the previous subpart.

Consider the function `max_value`, which computes the maximum value of an array using multithreading:

`max_value`: Computes the maximum value of an array		
Arguments	`uint32_t *arr`	An array of signed 32-bit integers.
	`int n`	The number of elements in the array.
Return value	`uint32_t`	The largest value in `arr`

```
1  uint32_t max_value(uint32_t* arr, int n) {
2    int global_max = 0;
3    omp_set_num_threads(97);
4    #pragma omp parallel
5    {
6        uint32_t local_max = 0;
7        int thread_id =  omp_get_thread_id();
8        int num_threads = omp_get_num_threads();
9        for (int i = thread_id; i < n; i += num_threads) {
10           if (arr[i] > local_max) {
11               local_max = arr[i];
12           }
13       }
14       #pragma omp critical
15       {
16           if (local_max > global_max) {
17               global_max = local_max;
18           }
19       }
20   }
21   return global_max
22 }
```

Q6.10 (2 points) Upon running this code, you notice that the speedup factor you attain as `n` approaches infinity is noticeably less than 100. Which of the following are probable causes for this? Select all that apply.

- [A] The number of threads is not a multiple of 4
- [■] The number of threads is more than the number of cores on the laptop
- [C] Interleaved memory accesses cause coherence misses
- [D] The critical segment creates an unparallelizable serial portion
- [E] None of the above

Solution:

- The number of threads not being a multiple of 4 should not significantly affect things, since the usual reasons for multiples of 4 being relevant (alignment issues, mostly) do not apply here.
- If the number of threads exceeds the number of cores on the machine you're running the code on, then only a few threads will be able to run at a time; the rest of the threads will be put to sleep while the OS assigns other threads, so you get speedup roughly proportional to the number of cores.
- While this interleaved access pattern would normally cause coherence misses, in this case, the array is only ever read. As such, cache blocks are safely shared between multiple cores without causing coherence misses.
- The critical segment in this case is after the main for loop, so as `n` approaches infinity, the effect of the critical segment vanishes to zero.

Q7 *Virtual Virtue-all* ✨ **(10 points)**

For questions Q7.1—Q7.3, suppose that we have a 64 GiB byte-addressable physical memory space, a 128 GiB virtual memory space, 1 MiB pages, and 32-bit page table entries.

Q7.1 (1 point) How many physical addresses do we have? Express your answer as a power of 2.

2^{36} addresses

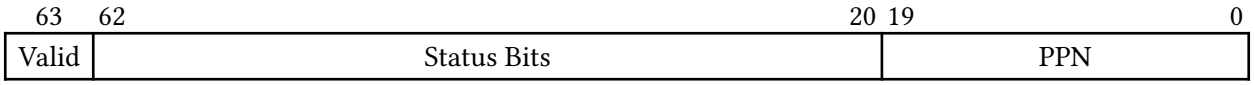
Q7.2 (2 points) Calculate the number of page offset bits, VPN bits, and PPN bits.

VPN: 17 bit(s) | PPN: 16 bit(s) | Offset: 20 bit(s)

Q7.3 (1 point) How many pages do we need to store the page table?

4 page(s)

For the remaining parts, assume we have 24-bit VPNs, 20-bit PPNs, 12-bit page offsets, and 64-bit page table entries, as formatted below:



Additionally, the TLB and the first four entries of the page table are shown below:

TLB		
Valid?	**VPN**	**PPN**
1	0x000000	0x0061C
0	0x000001	0x00700
0	0x000002	0x10502
0	0x000003	0x01620

Page Table
0x BE51 C1A5 5150 061C
0x C41B B100 0000 1620
0x 061C 051A FF04 A111
0x C41B B100 0001 5100
…

For each of the following virtual addresses, translate it to its corresponding physical address and answer whether accessing it will result in a TLB hit, TLB miss and page table hit, or a page fault. Assume each access occurs **independently**, not sequentially.

Q7.4 (2 points) `0x00000C16`

`0x0061CC16`

● TLB Hit Ⓒ TLB Miss & Page Table Hit Ⓑ Page Fault

Q7.5 (2 points) `0x0000200F`

`0x0B1A400F`

Ⓐ TLB Hit Ⓒ TLB Miss & Page Table Hit ● Page Fault

Q7.6 (2 points) `0x00003505`

`0x15100505`

Ⓐ TLB Hit ● TLB Miss & Page Table Hit Ⓑ Page Fault

Q8 吹雪にも蝉の鳴き声聞こえるか **(0 points)**

These questions will not be assigned credit. Feel free to leave them blank.

Q8.1 Write a haiku in a language that is not English. We will compile all valid haikus and publish them to the class when solutions release.

Q8.2 If there's anything else you want us to know, or you feel like there was an ambiguity in the exam, please put it in the box below.

For ambiguities, you must qualify your answer and provide an answer for both interpretations. For example, "if the question is asking about A, then my answer is X, but if the question is asking about B, then my answer is Y". You will only receive credit if it is a genuine ambiguity and both of your answers are correct. We will only look at ambiguities if you request a regrade.